

Ex.No : 2**Date:19/8/25**

STUDY OF SOCKET PROGRAMMING IN PYTHON

Aim:

To study the concepts of socket programming in python.

Introduction:

Socket definition:

A network socket is an endpoint of a two-way communication link between two programs or processes - client and server in our case - which are running on the network. This can be on the same computer as well as on different systems which are connected via the network. Both client/server communicate with each other by writing to or reading from the network socket. The technical equivalent in reality is a telephone communication between two participants. The network socket represents the corresponding number of the telephone line, or a contract in case of cell phones.

Socket programming in python

- Python's core networking library is Socket Module.
- Python's socket module has both class-based and instances-based methods.
- Class-based method is an intuitive approach which doesn't need an instance of a
- socket object.
- For example, in order to print a machine's IP address, you don't need a socket
- object. Instead, just call the socket's class-based methods.
- In instance-based method, if some data needed to be sent to a server application,
- it is more intuitive to create a socket object to perform that explicit operation.
- This module has everything you need to build socket servers and clients.

TCP Client-Server Communication Using Socket programming in python:

Algorithm:

Server:

1. Create a TCP socket using ``socket.socket(AF_INET, SOCK_STREAM)``.
2. Bind the socket to ``localhost`` and a specific port using ``bind()``.
3. Listen for incoming client connections using ``listen()``.
4. Accept a connection, receive the client's message using ``recv()``, and send a reply using ``send()``.
5. Ask the user whether to continue; if not, close the socket using ``close()``.

Client:

1. Create a TCP socket using ``socket.socket(AF_INET, SOCK_STREAM)``.
2. Connect the socket to the server using ``connect(('localhost', 55555))``.
3. Send a message to the server using ``send()``.
4. Receive the reply from the server using ``recv()`` and display it.
5. Close the socket connection using ``close()``.

Program Code:

Server:

```
import socket

sockfd=socket.socket(socket.AF_INET, socket.SOCK_STREAM)

print('Socket Created')

sockfd.bind(('localhost',55555))

sockfd.listen(3)

print('Waiting for connections')

while True:

    clientfd,addr=sockfd.accept()

    receivedMsg=clientfd.recv(1024).decode()

    print("Connected with ",addr)

    print("Message Received from Client: ",receivedMsg)

    clientfd.send(bytes(receivedMsg,'utf-8'))

    print("Message reply sent to Client!")

    print("Do you want to continue(type y or n):")

    choice=input()

    if choice=='n':

        break
```

Client:

```
import socket

clientfd=socket.socket(socket.AF_INET, socket.SOCK_STREAM)

clientfd.connect(('localhost',55555))

name=input("Enter your message:")

clientfd.send(bytes(name,'utf-8'))

print("Message Received from Server: ",clientfd.recv(1024).decode())
```

Output:**Server:**

```
C:\Users\ponso\Documents>python server3.py
Socket Created
Waiting for connections
Connected with ('127.0.0.1', 60350)
Message Received from Client: Hello!
Message reply sent to Client!
Do you want to continue(type y or n):
n
```

Client:

```
C:\Users\ponso\Documents>python client3.py
Enter your message:Hello!
Message Received from Server: Hello!
```

UDP Client-Server Communication Using Socket programming in python:

Algorithm:

Server:

1. Create a UDP socket using `socket.socket(AF_INET, SOCK_DGRAM)`.
2. Bind the socket to 'localhost' and a specific port using `bind()`.
3. Wait to receive a message from a client using `recvfrom()`.
4. Display the received message and send it back using `sendto()`.
5. Ask the user whether to continue or close the socket using `close()`.

Client:

1. Create a UDP socket using ``socket.socket(AF_INET, SOCK_DGRAM)``.
2. Enter the message to send to the server.
3. Send the message using ``sendto()`` with the server's address and port.
4. Receive the server's reply using ``recvfrom()`` and display it.
5. Ask the user whether to continue or close the socket using ``close()``.

Program Code:

Server:

```
import socket

sockfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

print("UDP Socket Created")

sockfd.bind(('localhost', 8000))

print("Waiting for messages...")

while True:

    data, addr = sockfd.recvfrom(1024)

    print("Message Received from Client:", data.decode())

    sockfd.sendto(data, addr)

    print("Message reply sent to Client!")

    choice = input("Do you want to continue (y/n): ")

    if choice.lower() == 'n':

        break

sockfd.close()
```

Client:

```
import socket

clientfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

while True:

    msg = input("Enter message to send: ")

    clientfd.sendto(msg.encode(), ('localhost', 8000))

    data, addr = clientfd.recvfrom(1024)

    print("Message from Server:", data.decode())

    choice = input("Do you want to continue (y/n): ")

    if choice.lower() == 'n':

        break

clientfd.close()
```

Output:**Server:**

```
C:\Users\ponso\Documents>python server4.py
UDP Socket Created
Waiting for messages...
Message Received from Client: Hello!
Message reply sent to Client!
Do you want to continue (y/n): n
```

Client:

```
C:\Users\ponso\Documents>python client4.py
Enter message to send: Hello!
Message from Server: Hello!
Do you want to continue (y/n): n
```

Result:

Thus concepts of socket programming in python were studied.